МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
<БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ>

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №4

Специальность АС-68

Выполнил
А. В. Венско,
студент группы АС-68

Проверил
А. А. Крощенко,
ст. преп. кафедры ИИТ,
«___> _____2026 г.

Брест 2026

Цель: построить, обучить и оценить многослойный перцептрон (MLP) для решения задачи классификации

Вариант 2 Диагностика рака груди

- Breast Cancer Wisconsin
- **Задача:** определить, является ли опухоль злокачественной (бинарная классификация).
- **Архитектура:**
  - входной слой;
  - один скрытый слой с 16 нейронами (ReLU);
  - выходной слой с 1 нейроном (Sigmoid).
- **Эксперимент:** обучите модель с 32 нейронами в скрытом слое. Как изменились метрики precision и recall?

Выполнение:

Код программы

```
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.optim as optim

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix
)

torch.manual_seed(42)
np.random.seed(42)

print("   Запуск задачи классификации (Breast Cancer Wisconsin)")

data = pd.read_csv("breast_cancer.csv")

data = data.drop(
    columns=["id", "Unnamed: 32"],
    errors="ignore"
)

X = data.drop("diagnosis", axis=1).values
y = (data["diagnosis"] == "M").astype(int).values

print("Количество записей:", len(data))
print("Количество признаков:", X.shape[1])
print("Доброкачественные опухоли:", np.sum(y == 0))
print("Злокачественные опухоли:", np.sum(y == 1))

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

X_train = torch.tensor(X_train, dtype=torch.float32)
X_test = torch.tensor(X_test, dtype=torch.float32)

y_train = torch.tensor(
    y_train.reshape(-1, 1),
    dtype=torch.float32
)

y_test = torch.tensor(
    y_test.reshape(-1, 1),
    dtype=torch.float32
)

class MLP(nn.Module):
    def __init__(self, input_size, hidden_size):
        super().__init__()

        self.layers = nn.Sequential(
            nn.Linear(input_size, hidden_size),
            nn.ReLU(),
            nn.Linear(hidden_size, 1),
            nn.Sigmoid()
        )

    def forward(self, x):
        return self.layers(x)

def train_model(hidden_size, name):
    torch.manual_seed(42)

    model = MLP(X_train.shape[1], hidden_size)

    criterion = nn.BCELoss()
    optimizer = optim.Adam(
        model.parameters(),
        lr=0.001
    )

    for epoch in range(1, 201):
        model.train()

        output = model(X_train)
        loss = criterion(output, y_train)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        if epoch % 50 == 0:
            print(f"{name} | Эпоха {epoch}, Loss: {loss.item():.4f}")

    model.eval()

    with torch.no_grad():
        output = model(X_test)
        predictions = (output >= 0.5).int().numpy().flatten()

    actual = y_test.numpy().flatten()

    accuracy = accuracy_score(actual, predictions)
    precision = precision_score(actual, predictions)
    recall = recall_score(actual, predictions)
```

```
    f1 = f1_score(actual, predictions)

    matrix = confusion_matrix(actual, predictions)

    print(f"\n{name}")
    print(f"Accuracy:  {accuracy:.4f}")
    print(f"Precision: {precision:.4f}")
    print(f"Recall:    {recall:.4f}")
    print(f"F1-score:  {f1:.4f}")

    print("\nМатрица ошибок:")
    print(matrix)

    return {
        "Модель": name,
        "Accuracy": accuracy,
        "Precision": precision,
        "Recall": recall,
        "F1-score": f1
    }

print("\nОБУЧЕНИЕ МОДЕЛИ С 16 НЕЙРОНАМИ")
result_16 = train_model(16, "MLP с 16 нейронами")

print("\nОБУЧЕНИЕ МОДЕЛИ С 32 НЕЙРОНАМИ")
result_32 = train_model(32, "MLP с 32 нейронами")

print("\nСРАВНЕНИЕ РЕЗУЛЬТАТОВ")

results = pd.DataFrame([result_16, result_32])

print(results.to_string(
    index=False,
    float_format=lambda x: f"{x:.4f}"
))

print("\nИЗМЕНЕНИЕ МЕТРИК")

precision_change = result_32["Precision"] - result_16["Precision"]
recall_change = result_32["Recall"] - result_16["Recall"]

print(f"Изменение Precision: {precision_change:+.4f}")
print(f"Изменение Recall: {recall_change:+.4f}")

if precision_change > 0:
    print("Precision увеличилась.")
elif precision_change < 0:
    print("Precision уменьшилась.")
else:
    print("Precision не изменилась.")

if recall_change > 0:
    print("Recall увеличилась.")
elif recall_change < 0:
    print("Recall уменьшилась.")
else:
    print("Recall не изменилась.")
```

Результат:

Запуск задачи классификации (Breast Cancer Wisconsin)
Количество записей: 569
Количество признаков: 30
Доброкачественные опухоли: 357
Злокачественные опухоли: 212

ОБУЧЕНИЕ МОДЕЛИ С 16 НЕЙРОНАМИ

```
MLP с 16 нейронами | Эпоха 50, Loss: 0.3615
MLP с 16 нейронами | Эпоха 100, Loss: 0.2103
MLP с 16 нейронами | Эпоха 150, Loss: 0.1453
MLP с 16 нейронами | Эпоха 200, Loss: 0.1129

MLP с 16 нейронами
Accuracy:  0.9737
Precision: 0.9756
Recall:    0.9524
F1-score:  0.9639

Матрица ошибок:
[[71  1]
 [ 2 40]]

ОБУЧЕНИЕ МОДЕЛИ С 32 НЕЙРОНАМИ
MLP с 32 нейронами | Эпоха 50, Loss: 0.2990
MLP с 32 нейронами | Эпоха 100, Loss: 0.1537
MLP с 32 нейронами | Эпоха 150, Loss: 0.1044
MLP с 32 нейронами | Эпоха 200, Loss: 0.0812

MLP с 32 нейронами
Accuracy:  0.9825
Precision: 1.0000
Recall:    0.9524
F1-score:  0.9756

Матрица ошибок:
[[72  0]
 [ 2 40]]

СРАВНЕНИЕ РЕЗУЛЬТАТОВ
          Модель  Accuracy  Precision  Recall  F1-score
MLP с 16 нейронами    0.9737     0.9756  0.9524    0.9639
MLP с 32 нейронами    0.9825     1.0000  0.9524    0.9756

ИЗМЕНЕНИЕ МЕТРИК
Изменение Precision: +0.0244
Изменение Recall: +0.0000
Precision увеличилась.
Recall не изменилась.
```

Вывод: В ходе лабораторной работы была разработана и обучена нейронная сеть многослойный перцептрон (MLP) для диагностики рака груди на основе набора данных Breast Cancer Wisconsin. Были обучены и сравнены две модели с одним скрытым слоем, содержащим 16 и 32 нейрона соответственно. При увеличении количества нейронов значение Precision повысилось с 0,9756 до 1,0000, а Recall осталось неизменным и составило 0,9524. Также Accuracy увеличилась с 0,9737 до 0,9825, а F1-score — с 0,9639 до 0,9756. Таким образом, модель с 32 нейронами показала более высокое качество классификации, однако увеличение количества нейронов не повлияло на полноту обнаружения злокачественных опухолей: количество ложноотрицательных прогнозов в обеих моделях осталось равным двум.